Sonar debris detector: data split, model and runtime

Project report, `sonar_edge`

4 November 2026

Scope

This report covers the state of the `sonar_edge` project as of the date above. It documents (i) how the sonar frames are split into scene folds and why, with worked examples; (ii) the detector architecture, its training and the configurations evaluated; (iii) the 8-bit quantisation and the C runtime that executes it; and (iv) the results obtained to date, including the resolution and width ablations run on 3–4 November. The per-milestone notes `docs/M0`–`M4` in the repository contain the complete derivations; this report is the consolidated account.

Contents

1 Data **2**

2 Evaluation protocol **2**

2.1 Scene reconstruction 2

2.2 Worked example 3

2.3 Fold assignment 3

2.4 Development split and cross-validation 4

3 Detector **4**

3.1 Architecture 4

3.2 Targets and loss 5

3.3 Inference 5

3.4 Training 5

4 Integer inference **6**

4.1 Quantisation 6

4.2 C runtime 6

4.3 Verification 7

5 Results **7**

5.1 Development split 7

5.2 Cross-validation 7

5.3 Seed variability 8

6 Discussion **8**

7 Hardware validation and next steps **10**

8 Conclusions and open items **10**

1 Data

The dataset is the water-tank subset of the marine debris FLS collection [1]: 1 868 forward-looking sonar frames from an ARIS Explorer 3000, projected to Cartesian coordinates at 320×480 pixels (width $\times$ height), single channel, 8 bit, each with one to five Pascal-VOC bounding boxes (3 882 in total) in eleven classes. Ten classes are debris objects. The eleventh, *Wall*, marks the tank walls; it is kept as a detector class because a deployed system must not report walls as debris, but it is excluded from the scene matching of Section 2 because walls are static and link unrelated recordings.

The frames are the complete sonar images, not object crops. The same collection also provides `watertank-cropped` (2 364 object patches cut from these frames, used by the MCNN paper and by the predecessor study `sonar_cnn`, which showed that the crop size alone predicts the class there) and a `turntable-cropped` set of objects on a rotating table; neither is used here. Pixel-level semantic labels (values 0–11, background plus the eleven classes, LabelMe) exist for every frame and were fetched on 4 November for a possible segmentation extension (Section 7).

The frames come from a small number of recordings in which the sonar drifts slowly over a fixed arrangement of objects, so consecutive frames are near-duplicates. The published frame indices carry no temporal information: the median thumbnail correlation of index-adjacent pairs (0.757) equals that of random pairs (0.747), and 2% of adjacent pairs have fully matching box layouts, the same as random pairs. A split by frame therefore puts near-identical frames on both sides, and the test score measures memorisation of the recording. This is the leak found in the predecessor study (`sonar_cnn`) for cropped patches; whole-frame detection inherits it.

2 Evaluation protocol

2.1 Scene reconstruction

Because the recording order is lost, it was reconstructed from the annotations. For two frames A and B with Wall boxes removed, the *layout match* from A to B is

$$m(A \to B) = \frac{1}{|A|} \left| \{\, p \in A : \exists\, q \in B,\ \mathrm{cls}(q) = \mathrm{cls}(p),\ \mathrm{IoU}(p, q) \geq 0.5 \,\} \right|, \tag{1}$$

the fraction of boxes in A that have a same-class box in B at intersection-over-union of at least 0.5 in original pixel coordinates. Two frames are *linked* if and only if

$$m(A \to B) = 1 \quad \text{and} \quad m(B \to A) = 1, \tag{2}$$

i.e. every object in either frame has a counterpart in the other. Frames with no non-wall boxes are linked to nothing. *Scenes* are the connected components of the link graph over all $\binom{1868}{2}$ pairs (union–find). Linkage is transitive by construction: the first and last frames of a long recording need not match directly but are joined through the frames between them.

Both strict choices are needed. Requiring every box to match in both directions prevents chaining through one shared object. With Wall boxes included, or with the required fraction relaxed from 1 to 0.5, 1 686 of the 1 868 frames collapse into a single component. The rule as stated gives 191 scenes; the largest contain 201, 158, 158, 119 and 90 frames, and 118 scenes are single frames, mostly frames whose only boxes are walls. Random members of four scenes of different sizes were inspected; each is one physical arrangement seen under small changes of viewpoint and gain.

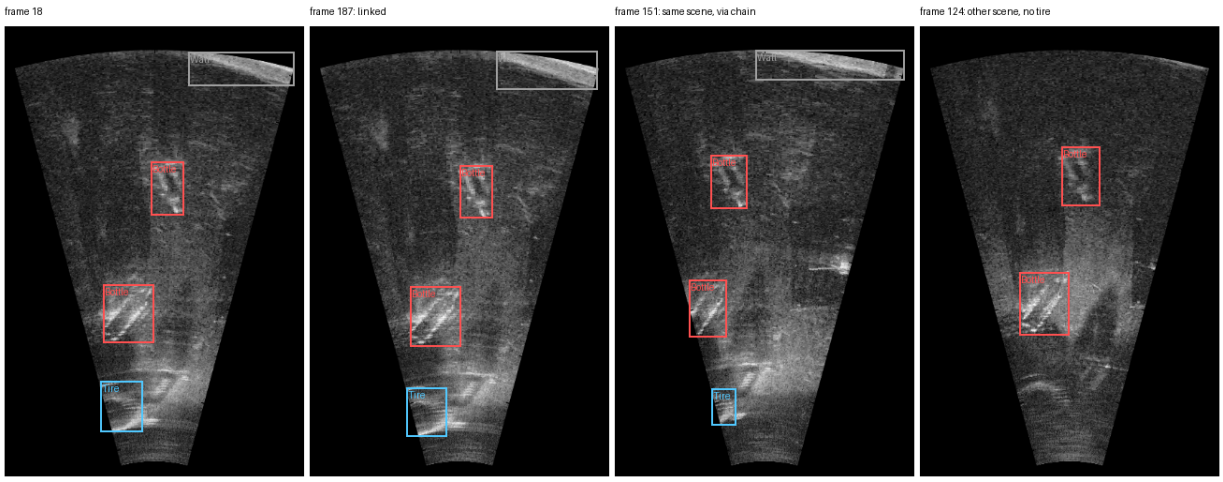


Figure 1: Frame 18 and three partner frames with their annotations (red: bottle, blue: tire, grey: wall). 18 and 187 are linked directly; 151 is in the same scene through intermediate frames; 124 is a different scene because the tire is absent.

Pair	Box in frame 18 (x, y, w, h)	Best same-class IoU	$m(18 \to B)$	$m(B \to 18)$	Outcome
18, 187	Bottle (156,144,35,57) Bottle (105,275,54,62) Tire (102,378,45,54)	0.71 0.85 0.76	3/3	3/3	linked
18, 151	Bottle (156,144,35,57) Bottle (105,275,54,62) Tire (102,378,45,54)	0.00 0.16 0.42	0/3	0/3	not linked directly; same scene (176, fold 4) through intermediate frames
18, 124	Bottle (156,144,35,57) Bottle (105,275,54,62) Tire (102,378,45,54)	0.56 0.71 –	2/3	2/2	not linked; scene 10, fold 3

Table 1: Layout-matching terms for the pairs of Figure 1. A pair is linked only when both fractions equal 1.

2.2 Worked example

Figure 1 shows frame 18 with three other frames and Table 1 the matching terms. Frames 18 and 187 are linked directly: each of the three objects matches at IoU 0.71–0.85. Frame 151 is in the same scene (scene 176, fold 4) but is not linked to frame 18 directly; the sonar has drifted until no box pair reaches IoU 0.5, and the link runs through intermediate frames. Frame 124 contains the same two bottles at almost the same positions but no tire, so $m(18 \to 124) = 2/3$ and the pair is not linked; it is in scene 10, which lies in fold 3.

The third pair shows the known weakness of the rule. Frame 124 is plausibly the same setup recorded before the tire was placed. The rule cannot tell, assigns the frame to a separate scene, and that scene lands in another fold, so some background texture is shared across the fold boundary. The error is in the conservative direction (too few links make the evaluation stricter, not more lenient), which is why the rule is used as it stands.

2.3 Fold assignment

The 191 scenes were assigned to $K = 5$ folds by a deterministic greedy procedure: scenes in order of decreasing size, each to the fold that minimises, after the assignment, the sum over classes of the squared share of that class's boxes plus the squared share of all frames. The assignment is

stored in `splits/scene_folds_v1.json` and is fixed for all experiments. Table 2 gives the fold sizes and class shares. Balance is limited by the data: several classes occur almost entirely in one scene (94% of standing-bottle boxes, 66% of valve, 54% of drink-carton), and a scene is never divided. Fold 3 is the smallest because it carries the standing-bottle scene.

	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4
Scenes	34	39	38	43	37
Frames	439	388	383	230	428
Bottle	43%	12%	6%	3%	36%
Can	22%	18%	24%	3%	33%
Chain	14%	15%	22%	37%	11%
Drink-carton	24%	54%	6%	2%	14%
Hook	23%	30%	13%	6%	28%
Propeller	32%	23%	22%	5%	18%
Shampoo-bottle	7%	10%	35%	5%	42%
Standing-bottle	3%	0%	3%	94%	0%
Tire	34%	39%	9%	3%	15%
Valve	2%	17%	66%	5%	10%
Wall	17%	25%	16%	18%	24%

Table 2: Fold composition: scenes and frames per fold, and the share of each class's boxes.

2.4 Development split and cross-validation

Development used one split: fold 0 as test set, fold 1 as validation set for epoch selection, folds 2–4 for training (1 041 / 388 / 439 frames).

Final numbers come from five-fold cross-validation. For $k = 0, \ldots, 4$ a model is trained from scratch with fold k as test set, fold $(k + 1) \bmod 5$ as validation set and the other three folds as training set; for $k = 2$, for example, folds 4, 0 and 1 train, fold 3 selects the epoch and fold 2 is tested. Every scene is tested exactly once, by a model that has seen none of its frames. The five test prediction sets partition the dataset and are scored together as one detection problem (*pooled* average precision, VOC convention). Pooling is the only estimate that is defined for classes concentrated in one scene: standing-bottle is testable only in the run that holds out fold 3, whose training set contains four instances, and pooling reports that precision without averaging over four undefined per-fold values. The mean and standard deviation of the per-fold mAP are given alongside as a measure of fold-to-fold variation. Both the float model and its 8-bit quantisation are evaluated this way, since the C runtime reproduces the latter exactly (Section 4).

GPU training is not bit-reproducible. A seed-identical repeat of the deployed configuration on the development split moved the test mAP by 3.8 points. Single-split differences of that size are therefore not interpreted, and configurations are compared only under cross-validation.

3 Detector

3.1 Architecture

The detector is a single-stage, anchor-free network in the CenterNet style [2], limited to operations that a minimal integer runtime can execute: a stack of 3×3 convolutions, each followed by batch normalisation and ReLU, with no residual connections or upsampling, and a 1×1 output convolution. The output stride is 8: a 480×320 input gives a 60×40 grid, a 240×160 input a 30×20 grid. Dilation in the later layers raises the receptive field to about 255 input pixels without further downsampling. Table 3 lists the configurations used.

Configuration	Layers (channels; s2 = stride 2, dk = dilation k)	Input	Grid	Params	MACs	RF
`small`	16s2, 32s2, 32s2, 32, 32d2, 32d4	240×160	30×20	42,463	34.8 M	254
`small`, scale 4/3	as above	360×240	45×30	42,463	78.4 M	254
`small_wide`	16s2, 32s2, 64s2, 64, 64d2, 64d4	240×160	30×20	135,359	90.4 M	254
`base`	16s2, 32s2, 48s2, 48, 48d2, 48d4, 48d8	480×320	60×40	102,831	283.7 M	255
`wide`	16s2, 32s2, 64s2, 64, 64d2, 64d4, 64d8	480×320	60×40	172,351	450.2 M	255

Table 3: Network configurations. Each layer is a 3×3 convolution with batch normalisation and ReLU (stride 1, dilation 1 unless marked); the head is a 1×1 convolution to $C + 4 = 15$ channels. RF is the receptive field in pixels of the full-resolution frame; MACs are multiply–accumulates per frame. `small` is the deployed configuration.

Per grid cell the head outputs $C = 11$ class logits, the box size (w, h) in units of the stride, and the centre offset $(\delta_x, \delta_y) \in [0, 1)$ within the cell. The class-logit bias is initialised to -2.19 (initial probability ≈ 0.01), following [2]. The input is the 8-bit frame scaled to $[0, 1]$; the half-resolution configurations average 2×2 pixel blocks, and the 4/3 scale uses a box-filter resize.

3.2 Targets and loss

For each ground-truth box with centre (c_x, c_y) and size (w, h) in grid units, a Gaussian is drawn into the target heatmap of its class, centred on the cell $(\lfloor c_x \rfloor, \lfloor c_y \rfloor)$, with the radius of [3]: the largest displacement at which the displaced box still has IoU ≥ 0.7 with the original. Overlapping Gaussians are merged by maximum. Size and offset targets are set at the centre cell only. The loss is

$$\mathcal{L} = \mathcal{L}_{\text{heat}} + 0.1\, \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{off}}, \tag{3}$$

with $\mathcal{L}_{\text{heat}}$ the penalty-reduced pixelwise focal loss of [2] ($\alpha = 2$, $\beta = 4$, normalised by the number of objects) on the sigmoid of the logits, and the two regression terms L1 losses at the object cells, normalised the same way.

3.3 Inference

Detections are the local maxima of the logit heatmap: a cell is kept if its value is greater than or equal to those of its 3×3 neighbours (a max-pooling comparison; no non-maximum suppression). The $K = 32$ highest peaks over all classes are taken, those below the logit threshold -4 (probability ≈ 0.018) dropped, and each remaining peak at cell (i_x, i_y) is converted to a box by

$$c_x = (i_x + \delta_x)\, s, \qquad c_y = (i_y + \delta_y)\, s, \qquad \left[c_x - \tfrac{ws}{2},\ c_y - \tfrac{hs}{2},\ c_x + \tfrac{ws}{2},\ c_y + \tfrac{hs}{2}\right], \tag{4}$$

where s is the stride. The decoder works on logits rather than probabilities; since the sigmoid is monotone, the integer runtime can apply the threshold to its raw head output.

3.4 Training

AdamW (weight decay 10^{-4}), one-cycle learning-rate schedule with peak 2×10^{-3} and 10% warm-up, batch size 16, seed 0, 150 epochs. The epoch with the highest validation mAP is kept and the test fold evaluated once. Augmentation: horizontal flip, integer translation of up to 8 pixels, random scaling by a factor in $[0.75, 1.3]$ about a random point with probability 0.7 (boxes scaled, clipped, and dropped when less than 30% of their area or 4 pixels remain), additive Gaussian noise with standard deviation drawn from $U[0, 0.04]$, multiplicative gain in $[0.7, 1.3]$ and additive bias in $[-0.1, 0.1]$. On the development split the scaling and noise terms were worth about 5 mAP points; lengthening the schedule without them changed nothing.

The metric is average precision at IoU 0.5 with all-point interpolation, per class, averaged over the classes present in the evaluated set. The implementation (`metrics.py`) is reused unchanged to score the C runtime.

4 Integer inference

4.1 Quantisation

Batch normalisation is folded into the preceding convolution, $W' = W\gamma/\sqrt{\sigma^2+\epsilon}$ and $b' = \beta - \mu\gamma/\sqrt{\sigma^2+\epsilon}$ per output channel, so the runtime has no normalisation layer. The scheme of [4] is used in its symmetric, zero-point-free form. Every activation is either an input pixel or a ReLU output, hence non-negative, and is stored as an unsigned 8-bit integer with zero point 0 and one scale s_a per layer. Weights are signed 8-bit with one scale per output channel, $s_{w,c} = \max|W'_c|/127$. With input scale s_{in} and output scale s_{out}, a layer computes per output channel c

$$\text{acc} = \sum_i W_{q,c,i}\, a_{q,i} + b_{q,c}, \qquad b_{q,c} = \text{round}\Big(\frac{b'_c}{s_{w,c}\, s_{\text{in}}}\Big), \qquad \text{out}_q = \text{clip}\Big(\text{round}\big(\text{acc}\cdot\tfrac{s_{w,c}\, s_{\text{in}}}{s_{\text{out}}}\big),\, 0,\, 255\Big), \tag{5}$$

where the clip is the ReLU. The real multiplier is replaced by an integer $M_c \in [2^{30}, 2^{31})$ and a shift n_c with $M_c 2^{-n_c}$ within a relative error of 10^{-8}; requantisation is then a 64-bit multiply, an add of 2^{n_c-1} (round half away from zero) and a shift. The 1×1 head is not requantised: its 32-bit accumulators are returned with their per-channel real scale and decoded directly. Activation scales are calibrated as the 99.99th percentile of the post-ReLU outputs over 64 training frames, divided by 255.

A reference implementation evaluates the integer network with double-precision tensor operations. This is exact: every product is below 2^{15} and every sum has fewer than 2^{12} terms, so all intermediates are integers far below 2^{53}. The reference is therefore a bit-exact specification of the runtime that is fast enough to score a whole test fold; it also exports the weight tables and three golden input–output pairs.

4.2 C runtime

The runtime is C11 with the standard library only, built with GNU make under `-std=c11 -O2 -Wall -Wextra -Werror -pedantic`. It is about 165 lines plus a generated header with the 8-bit kernels, 32-bit biases, the (M_c, n_c) pairs, the layer table and the tensor geometry as preprocessor constants. All memory is static: two activation buffers that alternate as input and output, and the 32-bit head map. The `small` weights occupy 42 KB.

Activations are stored HWC (channels innermost) with a zero border equal to the dilation d of the consuming layer, so that a 3×3 convolution with stride s reads, for output (o_y, o_x) and tap (k_y, k_x), the padded pixel $(o_y s + k_y d,\ o_x s + k_x d)$ without bounds checks, reproducing `padding = dilation` in PyTorch. Each tap is a dot product over c_{in} contiguous activation bytes and c_{in} contiguous weights. Accumulation is 32-bit ($|\text{acc}| < 2^{28}$), requantisation 64-bit; only non-negative quantities are shifted, so the implementation-defined right shift of a negative integer in C is never used.

The decoder scans each class's 32-bit logit map for cells greater than or equal to their existing 3×3 neighbours (the same as `max_pool2d` with its implicit $-\infty$ padding), converts the score to a real value in the same operation order as the Python reference, and forms the box in double precision with the same expression order as the Python decoder. These per-detection scalars are the only floating-point operations in the runtime. The interface is

```
int se_detect(const uint8_t *frame, float thr, se_detection_t *out, int max_out);
```

```
const int32_t *se_forward(const uint8_t *frame);   /* int32 head map, HWC */
```

and a command-line tool reads 8-bit PGM frames and prints one detection per line.

4.3 Verification

Two checks tie the C runtime to the Python reference. The golden test compares the full 32-bit head map from `se_forward` on the three exported frames with the reference and requires equality on every one of the $30 \times 20 \times 15$ values; it passes. The end-to-end test runs the compiled binary over the whole development test fold and scores its output with the same metric and ground truth as the Python models: 65.17% mAP in C against 65.16% for the Python reference, the residual coming from the order of detections with exactly equal scores. The accuracy of the deployed system is therefore established by the cross-validation of Section 5; only speed and energy remain to be measured on the device.

On one core of a laptop CPU (AMD Ryzen 5 5600H, scalar code, `-O2`) the deployed configuration takes 10 ms per frame, about 3.5 GMAC/s. A Raspberry Pi 4 is expected to need 35–50 ms per frame; this measurement awaits hardware.

5 Results

5.1 Development split

Table 4 gives the development-split results used to choose the configuration. They are single runs and carry the run-to-run variation of Section 2; the last row is a seed-identical repeat of the deployed configuration.

Configuration	Input	MACs	Val. mAP	Test mAP, float	Test mAP, int8 / C
`base`	480×320	284 M	66.1	63.1	–
`small` (deployed)	240×160	35 M	55.0	64.1	65.2 / 65.17
`wide`	480×320	450 M	67.8	68.8	–
`small`, repeat	240×160	35 M	59.9	60.3	–

Table 4: Development-split results (mAP@0.5 in %; fold 0 test, fold 1 validation; 150 epochs, strong augmentation).

5.2 Cross-validation

Table 5 gives the pooled cross-validation results and the per-fold test mAP of each configuration; Table 6 the pooled per-class average precision. All runs: 150 epochs, strong augmentation, seed 0, same folds.

Configuration	MACs	Pooled, float	Pooled, int8	Fold mean (s.d.)	Per-fold float, folds 0–4
`small` (deployed)	35 M	49.4	49.7	56.4 (8.9)	60.3, 59.3, 49.9, 44.9, 67.4
`small`, 360×240	78 M	54.6	53.7	59.5 (9.3)	64.0, 66.5, 52.3, 47.0, 67.7
`small`, 360×240, seed 1	78 M	52.2	52.8	58.1 (9.6)	61.0, 66.2, 49.2, 46.7, 67.5
`small`, 480×320	139 M	54.1	53.9	61.6 (9.8)	64.9, 65.8, 57.6, 47.0, 72.7
`small_wide`	90 M	50.2	49.6	58.0 (11.9)	60.1, 63.3, 51.1, 42.2, 73.4
`wide`	450 M	55.2	56.2	62.8 (8.3)	66.9, 68.6, 55.7, 52.1, 70.5

Table 5: Five-fold cross-validation (mAP@0.5 in %, pooled over all 1 868 frames). Seed 0 unless stated.

Class (boxes)	small	small, int8	small 360×240	small 480×320	small_wide	wide	wide, int8
Bottle (468)	78.0	77.4	84.2	83.2	55.9	84.7	82.7
Can (292)	9.1	9.6	16.1	14.6	8.3	18.8	20.5
Chain (320)	32.5	32.3	41.4	35.4	38.3	29.8	30.2
Drink-carton (347)	62.4	64.1	69.8	80.6	69.2	81.4	79.8
Hook (171)	90.8	91.4	93.6	91.9	95.0	95.5	94.9
Propeller (197)	51.2	50.9	49.1	53.9	47.6	46.7	46.9
Shampoo-bottle (111)	27.7	25.5	37.6	35.8	42.0	30.5	31.4
Standing-bottle (65)	1.2	2.5	3.7	3.6	2.0	24.1	26.7
Tire (615)	86.8	86.8	84.4	93.5	84.0	93.9	92.7
Valve (240)	22.8	23.2	38.6	21.4	28.8	17.5	27.5
Wall (1056)	81.3	83.4	82.6	81.4	81.0	84.8	84.9
mAP, pooled	49.4	49.7	54.6	54.1	50.2	55.2	56.2

Table 6: Pooled per-class average precision (%) under five-fold cross-validation; float models unless marked int8.

5.3 Seed variability

Each cross-validation result above is one training run per fold. To measure the training noise under the full protocol, the deployed configuration and the 360×240 model were each run three times through all five folds with seeds 0, 1 and 2, everything else unchanged (Table 7; seed 0 is the run reported in Table 5).

	small, 240×160	small, 360×240
Pooled mAP, float, per seed	49.4, 47.4, 47.5	54.6, 52.2, 51.0
Pooled mAP, float, mean (s.d.)	**48.1 (1.1)**	**52.6 (1.9)**
Pooled mAP, int8, mean (s.d.)	48.1 (1.6)	52.3 (1.7)
Fold 0, mean (s.d.)	55.5 (5.4)	61.0 (3.0)
Fold 1	58.5 (0.7)	65.1 (2.2)
Fold 2	48.9 (1.0)	52.2 (3.0)
Fold 3	42.5 (2.7)	48.2 (2.4)
Fold 4	67.4 (0.2)	67.5 (0.2)
Bottle	73.7 (3.8)	85.4 (1.6)
Can	8.0 (1.8)	15.3 (0.7)
Chain	27.6 (8.0)	39.6 (3.3)
Drink-carton	64.7 (2.9)	63.1 (9.7)
Hook	89.0 (2.0)	90.5 (3.5)
Propeller	48.8 (2.4)	47.3 (1.7)
Shampoo-bottle	27.7 (5.2)	33.3 (6.7)
Standing-bottle	1.6 (1.0)	6.4 (2.9)
Tire	84.6 (2.9)	89.3 (4.3)
Valve	22.7 (1.1)	26.5 (11.3)
Wall	81.1 (0.3)	82.2 (0.5)

Table 7: Three seeds under the full five-fold protocol (mAP@0.5 and pooled per-class AP in %, mean and standard deviation over seeds; float unless marked int8).

6 Discussion

Protocol. Pooled cross-validation values are 10–15 points below the development-split values of the same configurations. The fold ranking is the same for every configuration (folds 0, 1 and 4 easier, folds 2 and 3 harder), so the per-fold spread is a property of the data, and the development

fold is one of the easier ones. The per-fold standard deviation of 8–12 points is therefore not a measure of model uncertainty. The relevant noise figures are the seed-identical repeat on one split (3.8 points) and the three-seed repeats of Table 7: the pooled mAP has a seed standard deviation of 1–2 points, because pooling averages five independently trained models, while single folds move by up to 5 points between seeds (fold 0 of `small`: 60.3, 56.6, 49.6). Single-fold values are therefore not interpreted on their own. The seed-0 runs reported in Table 5 are the best of their three seeds for both configurations; the seed means are 1.3 and 2.0 points lower. Per class, valve, drink-carton, chain and shampoo-bottle move by 5–11 points between seeds, more than between configurations, because their boxes lie in few scenes; bottle, can, hook, tire and wall have standard deviations below 4 and their differences are interpretable.

Resolution versus width. `wide` is 5.8 pooled points above `small` and is above it on every fold, by 3.1 to 9.3 points, which is outside the run-to-run variation. The gain sits in the small and low-contrast classes (drink-carton +19, standing-bottle +23, can +10, bottle +7, tire +7), whose extent at the half-resolution input is close to the 8-pixel stride. `wide` differs from `small` in input resolution and in channel width. `small_wide` isolates width: +0.8 pooled (float) and −0.1 (int8) against `small`, per-fold differences between −2.7 and +6.0, all inside the run-to-run variation. The unchanged `small` network at 360×240 (78 M MACs) isolates resolution: +4.5 pooled on the three-seed means (52.6 against 48.1), above `small` on every fold's mean, and for every one of the nine seed pairings (its worst seed, 51.0, exceeds the best `small` seed, 49.4); its seed-0 run is below `wide` on every fold by 2 to 5 points. The improvement of `wide` is therefore due to input resolution, and 90% of it is available at a sixth of its cost, inside the budget of about 100 M MACs set for the target device. The gains occur in the same classes (bottle, can, drink-carton) and in addition in chain, shampoo-bottle and valve; standing-bottle, the class with the smallest median extent, improves only with `wide`. The same network at full resolution (480×320, 139 M MACs) pools 54.1, no better than 360×240 (−0.5; per-fold +0.9, −0.7, +5.3, 0.0, +5.0) and 1.1 below `wide`, so the resolution curve of the `small` network is 49.4, 54.6, 54.1 for 35, 78, 139 M MACs: it saturates at 360×240, and the last point of `wide` is what its doubled channel width buys at full resolution.

Quantisation. Post-training 8-bit quantisation is neutral for `small` and `wide` (pooled +0.3 and +1.0). For `small_wide` and for `small` at 360×240 it costs 0.2–3.6 points on every fold (−0.6 and −0.9 pooled). A per-layer comparison of the dequantised activations with the float activations on 48 test frames per fold shows no difference between the configurations: relative RMS error of 3–8% per layer, saturation below 0.02%, and head-logit errors of 0.04–0.07 at the float model's peaks with a bias of −0.01, for `small`, `small_wide` and the 360×240 model alike. The per-class differences are dominated by shampoo-bottle, which under int8 moves by up to 30 AP points on single folds for every configuration, including the two for which the pooled effect is positive; its 111 boxes lie in few scenes. Over three seeds the int8 pooled means equal the float means within 0.3 points for both configurations and the per-seed sign varies (+0.3, −0.7, +0.3 and −0.9, +0.6, −0.6), so the consistent negative sign of the first 360×240 seed was chance; quantisation is neutral within seed noise for all configurations.

Low-performing classes. Three classes stay well below the rest under every configuration, for different reasons. *Standing-bottle* (65 boxes) has 94% of its instances in one scene; under the scene-fold protocol that scene is the test set of one run whose training set holds four instances, so its AP (1–10) measures the dataset, not the model, and a random frame split would hide this. *Can* (292 boxes, median 40×40 px) and *drink-carton* (27×30 px) are the smallest objects; at half resolution they approach the 8-pixel output stride, and raising the input to 360×240 lifts can from 9 to 16 and drink-carton from 62 to 70. *Valve* (240 boxes, 66% in one scene) and *shampoo-bottle* (111 boxes) rest on few scenes and move by 15–30 points between seeds (Table 7), so their

single-run values carry little information. The measures available, in order of cost: (i) sampling frames that contain rare classes more often, or weighting the focal loss per class; (ii) copy–paste augmentation, pasting annotated rare objects (with their masks, which the dataset provides) into other frames, which is plausible for sonar because objects are compact on a dark background; (iii) an output stride of 4 instead of 8 for the small classes, at about twice the compute. None of these can create training instances of standing-bottle outside its scene; that class is reported as a limitation.

Pooled per-class values. Pooled AP depends on the score calibration of the individual fold models, since detections from all five are ranked on one list. The bottle AP of `small_wide` (55.9 against 78.0 for `small`) is an example: its per-fold bottle AP is 36, 78, 54, 84 and 84, and the one poorly calibrated fold-0 model depresses the pooled value by more than its share of the boxes. This is why the per-fold values are reported next to the pooled ones.

7 Hardware validation and next steps

Raspberry Pi (M5). The runtime is built on the device with `make` (gcc and make only) or cross-compiled with `make CC=aarch64-linux-gnu-gcc`. The plan: (i) `make test` on the Pi, which must reproduce the golden head maps bit for bit, as the arithmetic is integer-only and platform-independent; (ii) latency with `sonar-detect -n` over the 439 development test frames, reporting mean and 95th percentile per frame for the deployed 240×160 model and the 360×240 candidate, single core, then with the optional NEON inner loop; (iii) energy with an inline USB-C power meter (or an INA219 on the 5 V rail) sampled at 1 Hz, as the difference between loaded and idle power divided by the frame rate, giving joules per frame, plus the idle baseline of the board. Expected values from the laptop measurement are 35–50 ms per frame for the deployed model and 80–110 ms for 360×240 on a Pi 4; these are estimates until measured. The hardware was not available at the time of writing.

Segmentation. The pixel labels make a multi-task extension possible at almost no runtime cost: a second 1×1 head of $C + 1$ channels on the same stride-8 features, trained with per-cell cross-entropy against the downsampled mask, evaluated as mean IoU at full resolution after nearest upsampling, and executed in the runtime as one `argmax` per cell. Whether the shared features also improve the boxes is an empirical question for the same cross-validation protocol.

8 Conclusions and open items

- The scene-fold protocol is in place and fixed. It removes the near-duplicate leak of random splits at the price of large fold-to-fold variation, which is a property of the dataset.
- The deployed configuration, `small` at 240×160, reaches $48.1 \pm 1.1\%$ pooled mAP over three seeds (48.1% quantised) at 35 M MACs and 10 ms per frame on a laptop core. The C runtime is bit-exact with the Python int8 reference and scores identically on the test fold.
- Input resolution, not channel width, is what the larger reference model gains from, and the gain saturates at 360×240 for this network. `small` at 360×240 gives $52.6 \pm 1.9\%$ over three seeds (52.3% quantised) against $48.1 \pm 1.1\%$ for the deployed model, at 78 M MACs, and is the recommended replacement if the latency measured on the target device allows it; deployment needs only a regenerated weight header, since the runtime takes its geometry from that header. Expected cost: about 22 ms per frame on the laptop, 80–110 ms on a Pi 4.

- Open: seed repeats of `wide`, `small_wide` and the 480×320 run, which have one seed each (about ± 2 points); the latency and energy measurement on the Raspberry Pi (M5), for which the runtime cross-compiles with `make CC=aarch64-linux-gnu-gcc`.
- Known limits of the evaluation: three seeds for two configurations, one for the others; standing-bottle is evaluable only under one fold assignment with four training instances; Wall is scored like the other classes although its boxes are large and loosely defined; the scene rule errs toward too few links, as the example of Figure 1 shows, and the residual background leak across folds has not been quantified.

References

[1] M. Valdenegro-Toro, D. C. Padmanabhan, D. Singh, B. Wehbe and Y. Petillot, *Marine debris forward-looking sonar datasets*, 2025. Repository `mvaldenegro/marine-debris-fls-datasets`, commit `4a77998`.

[2] X. Zhou, D. Wang and P. Krähenbühl, "Objects as points," arXiv:1904.07850, 2019.

[3] H. Law and J. Deng, "CornerNet: Detecting objects as paired keypoints," in *ECCV*, 2018.

[4] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *CVPR*, 2018.